

Java Constants

Using the `final` keyword for unchangeable data

1. The "final" Rule

A constant is a variable that is **read-only**. Once you assign a value to it, that value is locked forever. In Java, we use the `final` keyword to achieve this.

```
final int MAX_USERS = 100;  
// MAX_USERS is now locked at 100
```

2. Prevention of Errors

If you accidentally try to change a constant later in your code, Java will stop you with a **Compiler Error**. This makes your programs much safer.

Attempting to change:

```
MAX_USERS = 200;
```

→ ✗ Error: cannot assign a value to final variable 'MAX_USERS'

3. Naming Conventions

To help other programmers immediately identify constants, Java developers use a special naming style:

ALL_CAPS_WITH_UNDERSCORES

Example: MINUTES_PER_HOUR instead of minutes

Codehive

4. Constant vs. Variable

Normal Variable

Can be updated as the program runs (e.g., current score).

```
int score = 0;  
score = 10; // OK
```

Constant (final)

Fixed values that never change (e.g., PI, Birth Year).

```
final int YEAR = 2025;  
YEAR = 2026; // ❌ ERROR
```

5. Real-World Applications

Use constants for values that are "facts" within your application:

```
final double PI = 3.14159;  
final int HOURS_IN_DAY = 24;  
final String COMPANY = "CodeHive";
```



Quick Summary

- Adding `final` makes a variable "constant" (read-only).
- You **must** assign a value when creating it.
- Use **UPPER_CASE** names for clear identification.
- Constants prevent accidental bugs and make code easier to maintain.
- Ideal for math values, limits, and configuration settings.